30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 19

Security — Part 1: Threat Modeling, Keystore, Permissions, Intents & WebView



Esmaeeil Moradi
@esmaeeil-moradi700

Why security is Day 19

Security questions test whether you can see your app **through an attacker's eyes**. Since last year's edition, the ground has moved:

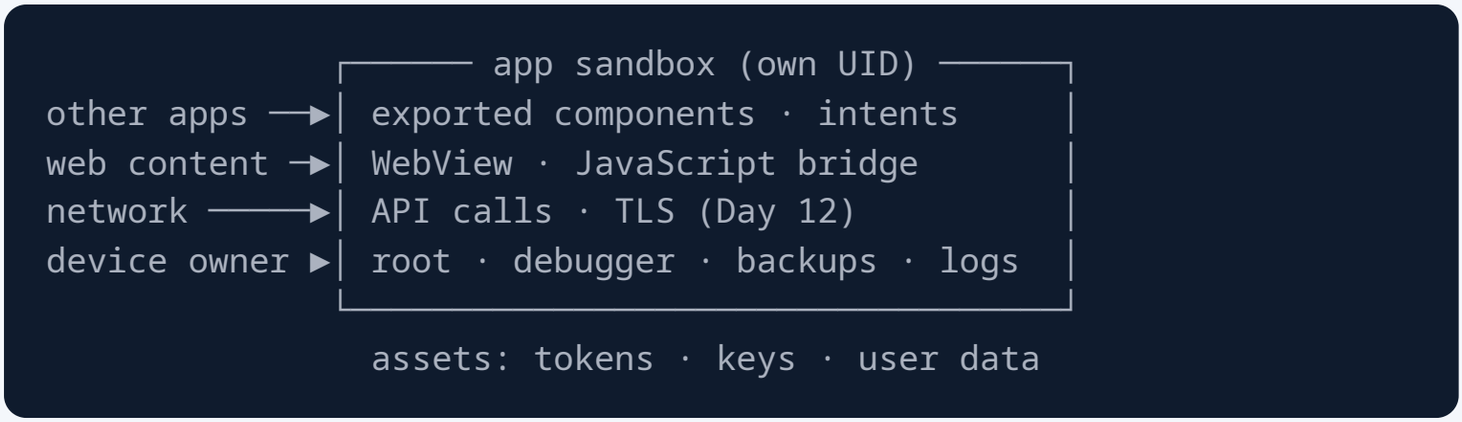
- Jetpack Security (`EncryptedSharedPreferences`, `EncryptedFile`) is **deprecated** — you must know the Keystore directly
- Every Android release tightened defaults: explicit `exported`, immutable `PendingIntent`s, partial photo access, restricted implicit intents
- Google Play enforces permission policies and a Data safety form that includes your SDKs
- Apps now hold refresh tokens, health records and payments — the stakes are real

Level	Questions
Junior	The platform security model · Where data lives · Runtime permissions
Mid	Media & partial access · Exported components & intents · PendingIntent · WebView
Senior	Android Keystore · Storage after security-crypto · Leaks: logs, backups, screens
Architect	Threat modeling · SDKs & the permission blast radius · Secure-by-default platform



Esmaeeil Moradi
@esmaeeil-moradi700

Mental model: the sandbox has doors



- The sandbox protects your **files** from other apps — not your **entry points**
- Everything that crosses a door is **untrusted input**: extras, deep links, web pages, SDK callbacks
- Assume the **device owner** can be the attacker: protect keys and decide on the server

START FROM ASSETS, NOT FROM LIBRARIES

"What is worth stealing, who wants it, and through which door do they come in?" — answer that before choosing any crypto API.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 19

Junior

The platform security model · Where data lives · Runtime permissions



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. Android's security model in one table

Layer	What it gives you
App sandbox — one Linux UID per app, enforced by SELinux	Other apps can't read your private files or memory
App signing	Updates must come from the same key (or its rotation lineage)
Permissions	Sensitive data and APIs need an explicit grant
Verified Boot + file-based encryption	An unmodified OS; data at rest encrypted and tied to the lock screen
Android Keystore (TEE / StrongBox)	Key material that never enters your process

File-based encryption is mandatory on devices that launch with Android 10 or newer. The **principle of least privilege** ties it together: ask only for what the feature needs.



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. What the sandbox does *not* cover

INTERVIEW TRAP

"The sandbox makes my app safe." — It isolates **files and memory**. It does nothing for an exported Activity that trusts its extras, a WebView that loads attacker content, a token printed to logcat, or a user with root who opens your data folder.

INTERVIEW TRAP

"Internal storage is encrypted, so secrets are safe there." — File-based encryption protects a **powered-off or locked** device. While your app runs, the files are readable to root and, by default, included in **Auto Backup** (Q2).

FOLLOW-UP

"How do you decide what to protect?" — With a threat model: assets, attackers, entry points, mitigations. That's Q11.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Where data lives — and who can read it

Location	Who can read it	Use for
Internal storage (`filesDir`, DataStore, Room)	Your app only — and root	App data; high-value secrets **encrypted**
App-specific external (`getExternalFilesDir`)	Your app (other apps only on Android 9 and lower)	Large, non-sensitive files
Shared storage (MediaStore, Downloads)	The user and other apps	Files the user owns
Android Keystore	Nobody — keys are non-exportable	**Keys**, not data
Cloud backup / device transfer	Restored onto another device	Only what you allow

`MODE_WORLD_READABLE` throws a `SecurityException` for apps targeting Android 7.0+. Share files with a `FileProvider` and a temporary URI grant — never a `file://` path.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Backups: the forgotten copy of your data

```xml
<!-- res/xml/data_extraction_rules.xml (Android 12+) -->
<data-extraction-rules>
    <cloud-backup>
        <exclude domain="sharedpref" path="session.xml" />
        <exclude domain="file" path="datastore/" />
    </cloud-backup>
    <device-transfer>
        <exclude domain="sharedpref" path="session.xml" />
        <exclude domain="file" path="datastore/" />
    </device-transfer>
</data-extraction-rules>
```

Reference it with `android:dataExtractionRules`; keep `android:fullBackupContent` for Android 11 and lower.

INTERVIEW TRAP

"*`allowBackup="false"` turns backup off.*" — For apps targeting Android 12+, it disables **cloud** backup but **not device-to-device transfer**. Exclude sensitive files explicitly in the `<device-transfer>` section.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Runtime permissions, the modern way

```
@Composable
fun ScanButton(onScan: () -> Unit) {
    val context = LocalContext.current
    val launcher = rememberLauncherForActivityResult(
        ActivityResultContracts.RequestPermission()
    ) { granted -> if (granted) onScan() else showWhyNeeded() }

    Button(onClick = {
        val granted = ContextCompat.checkSelfPermission(
            context, Manifest.permission.CAMERA,
        ) == PackageManager.PERMISSION_GRANTED
        if (granted) onScan()
        else launcher.launch(Manifest.permission.CAMERA)
    }) { Text("Scan receipt") }
}
```

- Ask **in context**, at the moment the feature is used — not all at launch
- `shouldShowRequestPermissionRationale()` tells you when to explain first



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Permission types and the rules around them

Type	Granted	Examples
Install-time (normal)	Automatically	`INTERNET`, `VIBRATE`
Runtime (dangerous)	System dialog, by the user	`CAMERA`, `ACCESS_FINE_LOCATION`, `POST_NOTIFICATIONS`
Signature	Only apps signed with the same key	Your own custom permissions
Special (app op)	User toggle in Settings	`SYSTEM_ALERT_WINDOW`, `SCHEDULE_EXACT_ALARM`

- Android 11+: **one-time** grants for location, camera and microphone; permissions of **unused apps are auto-reset**

INTERVIEW TRAP

"If the user denies, ask again next launch." — From Android 11, after the user denies the same permission **twice**, the system stops showing the dialog; `launch()` returns `false` immediately. Explain the value, then link to app settings.

INTERVIEW TRAP

"A flashlight app needs `CAMERA`." — `CameraManager.setTorchMode()` needs no permission. Asking for `CAMERA` there is exactly the over-reach least privilege forbids.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 19

Mid-Level

Media & partial access · Exported components & intents · PendingIntent · WebView



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Photos and media: pick, don't read

```
val pickPhoto = rememberLauncherForActivityResult(
    ActivityResultContracts.PickVisualMedia()
) { uri: Uri? -> uri?.let(viewModel::onPhotoPicked) }

Button(onClick = {
    pickPhoto.launch(
        PickVisualMediaRequest(
            ActivityResultContracts.PickVisualMedia.ImageOnly
        )
    )
}) { Text("Attach lab result") }
```

Android	Media access
13 (API 33)	`READ_EXTERNAL_STORAGE` replaced by `READ_MEDIA_IMAGES` / `_VIDEO` / `_AUDIO`
14 (API 34)	The user can grant **selected photos only** (`READ_MEDIA_VISUAL_USER_SELECTED`)
Any	The **Photo Picker** needs no permission at all



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. What interviewers check

INTERVIEW TRAP

"I'll request `READ_MEDIA_IMAGES` *so the user can attach one photo."* — Google Play's photo and video permissions policy limits these permissions to apps whose **core feature** needs broad access. Everyone else uses the Photo Picker.

INTERVIEW TRAP

"Once granted, I have the gallery." — On Android 14 the user may pick **"Select photos"**. Re-check access every time the screen opens and offer a way to change the selection; the grant can change while your app is in the background.

FOLLOW-UP

"How long is a picked URI readable?" — By default until the device restarts or your app process stops. Copy the file into app storage, or call `takePersistableUriPermission()` if you really need long-term access.

FOLLOW-UP

"And location?" — Two steps: foreground first (the user may choose **approximate**, Android 12+), background later and separately — on Android 11+ the user grants it in Settings.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Exported components: every door you open

```
<!-- Only apps signed with our key may call it -->
<permission
    android:name="com.example.permission.SYNC"
    android:protectionLevel="signature" />

<service
    android:name=".sync.SyncService"
    android:exported="true"
    android:permission="com.example.permission.SYNC" />

<!-- Internal screen: unreachable from other apps -->
<activity
    android:name=".internal.DebugActivity"
    android:exported="false" />
```

- Targeting Android 12+, every component with an `<intent-filter>` must declare `android:exported` — or installation fails. Before that, a filter **silently** meant `exported="true"`.
- Exported means **any app** can start it with an explicit intent and **any extras**.

INTERVIEW TRAP

"The intent filter limits who can call it." — No. Filters route implicit intents; they are not access control. Use `exported="false"` or a `signature` permission.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Intent redirection and runtime receivers

```
val next = IntentCompat.getParcelableExtra(
    intent, "next", Intent::class.java)

// BAD: launches whatever Intent another app put inside
// startActivity(next)

// GOOD: forward only to our own, known screens
val target = next?.resolveActivity(packageManager)
if (next != null && target != null &&
    target.packageName == packageName &&
    target.className in allowedScreens
) {
    next.removeFlags(
        Intent.FLAG_GRANT_READ_URI_PERMISSION or
            Intent.FLAG_GRANT_WRITE_URI_PERMISSION
    )
    startActivity(next)
}
```

- **Intent redirection:** an exported Activity that forwards an embedded `Intent` lets any app reach your **non-exported** components through you
- Runtime receivers: `ContextCompat.registerReceiver(…, RECEIVER_NOT_EXPORTED)` — the flag is mandatory when targeting Android 14+. In debug, StrictMode's `detectUnsafeIntentLaunch()` flags redirection.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. PendingIntent: a token that acts as *you*

```
val intent = Intent(context, ReminderReceiver::class.java)
    .setAction(ACTION_TAKE_DOSE)
    .putExtra(EXTRA_REMINDER_ID, reminder.id)

val pending = PendingIntent.getBroadcast(
    context,
    reminder.id.hashCode(), // distinct per reminder
    intent,
    PendingIntent.FLAG_IMMUTABLE or
        PendingIntent.FLAG_UPDATE_CURRENT,
)
```

- The notification system, `AlarmManager` or a widget host later fires it **with your app's identity and permissions**
- Targeting Android 12+: `FLAG_IMMUTABLE` or `FLAG_MUTABLE` is **required**. Default to immutable.
- `FLAG_MUTABLE` only when the receiver must fill in data: inline reply, bubbles, geofencing callbacks
- Always an **explicit** intent. Targeting Android 14+, a mutable `PendingIntent` with an implicit intent throws



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. What interviewers check

INTERVIEW TRAP

"Mutable and implicit is fine — it's my own notification." — Whoever gets the token can fill in the empty parts: point it at your **non-exported** component or add URI grants. It then runs as your app.

FOLLOW-UP

"Why does every notification open the same item?" — Two `PendingIntent`s are the **same token** when the request code and the intent's action, data, type, component and categories match — **extras are ignored**. Use a distinct request code (or data URI) per item.

RULE OF THUMB

Explicit + immutable by default. Every `FLAG_MUTABLE` in a code review needs a one-line justification.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. WebView: settings and navigation

```
webView.settings.apply {
    javaScriptEnabled = true  // only if the page needs it
    allowFileAccess = false   // default when targeting 30+
    allowContentAccess = false
    mixedContentMode = WebSettings.MIXED_CONTENT_NEVER_ALLOW
}
webView.webViewClient = object : WebViewClientCompat() {
    override fun shouldOverrideUrlLoading(
        view: WebView, request: WebResourceRequest,
    ): Boolean {
        val url = request.url
        val trusted = url.scheme == "https" &&
            url.host in allowedHosts
        if (!trusted) openInBrowser(url)
        return !trusted // true = WebView won't load it
    }
}
```

- Local pages: `WebViewAssetLoader` serves them over `https://appassets.androidplatform.net` — no `file://` access needed
- Sign-in and payment pages: **Custom Tabs**, not a WebView (Day 12)



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. The JavaScript bridge

```
// Legacy: every page and frame in this WebView can call it
webView.addJavascriptInterface(Bridge(), "Android")

// Better: an allow-list of origins (androidx.webkit)
if (WebViewFeature.isFeatureSupported(
        WebViewFeature.WEB_MESSAGE_LISTENER)
) {
    WebViewCompat.addWebMessageListener(
        webView, "nativeBridge",
        setOf("https://help.example.com"),
    ) { _, message, _, isMainFrame, _ ->
        if (isMainFrame) viewModel.onWebMessage(message.data)
    }
}
```

INTERVIEW TRAP

"`addJavascriptInterface` *is safe since API 17."* — API 17 only stopped reflection: now just `@JavascriptInterface` methods are exposed. But they are exposed to **every** page and iframe the WebView ends up on — a redirect or an ad is enough.

INTERVIEW TRAP

`webView.loadUrl(intent.data.toString())` from a deep link — an attacker chooses what runs inside your app's WebView, bridge included. Allow-list hosts first.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 19

Senior

Android Keystore · Storage after security-crypto · Leaks: logs, backups, screens



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Android Keystore: create a key that never leaves

```
fun getOrCreateKey(alias: String): SecretKey {
    val ks = KeyStore.getInstance("AndroidKeyStore")
        .apply { load(null) }
    (ks.getKey(alias, null) as? SecretKey)?.let { return it }

    val spec = KeyGenParameterSpec.Builder(
        alias,
        KeyProperties.PURPOSE_ENCRYPT or
            KeyProperties.PURPOSE_DECRYPT,
    )
        .setBlockModes(KeyProperties.BLOCK_MODE_GCM)
        .setEncryptionPaddings(
            KeyProperties.ENCRYPTION_PADDING_NONE)
        .setKeySize(256)
        .build()
    return KeyGenerator.getInstance(
        KeyProperties.KEY_ALGORITHM_AES, "AndroidKeyStore",
    ).apply { init(spec) }.generateKey()
}
```

The `SecretKey` you get back is only a **handle**. Encryption runs inside the Keystore — in the TEE, or in StrongBox if you asked for it.



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Key options that matter

Option	Effect
Non-exportable (always)	Your process never sees the key bytes
`setIsStrongBoxBacked(true)` (API 28)	A separate secure chip; slower; **throws** if the device has none
`setUserAuthenticationRequired(true)`	Usable only after the user authenticates (Day 20)
`setUnlockedDeviceRequired(true)` (API 28)	Unusable while the device is locked
`KeyInfo.securityLevel` (API 31)	Where the key really lives: TEE, StrongBox or software

INTERVIEW TRAP

"*`setIsStrongBoxBacked(true)` uses hardware if available.*" — Last year's version said this, and it's wrong. Without StrongBox, key generation throws `StrongBoxUnavailableException`; catch it and fall back to the regular TEE-backed key.

INTERVIEW TRAP

"Keys in the Keystore can't be abused." — The **bytes** can't be extracted, but code running as your app — malware on a rooted device, a hooking tool — can still ask the Keystore to decrypt. That's why high-value keys are bound to user authentication.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Secure storage after security-crypto

Jetpack Security (`androidx.security:security-crypto`) is **deprecated** — `EncryptedSharedPreferences` and `EncryptedFile` included. The replacement isn't another wrapper: a **Keystore key + AES-GCM + the storage you already use**.

```
class KeystoreCipher(private val alias: String) {
    private fun key(): SecretKey = getOrCreateKey(alias) // Q8

    fun encrypt(plain: ByteArray): ByteArray {
        val c = Cipher.getInstance("AES/GCM/NoPadding")
        c.init(Cipher.ENCRYPT_MODE, key()) // Keystore picks IV
        return c.iv + c.doFinal(plain)     // [IV | ciphertext]
    }

    fun decrypt(blob: ByteArray): ByteArray {
        val c = Cipher.getInstance("AES/GCM/NoPadding")
        val spec = GCMParameterSpec(128, blob, 0, IV_SIZE)
        c.init(Cipher.DECRYPT_MODE, key(), spec)
        return c.doFinal(blob, IV_SIZE, blob.size - IV_SIZE)
    }

    private companion object { const val IV_SIZE = 12 }
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. A token store on DataStore

```
class TokenStore(
    private val dataStore: DataStore<Preferences>,
    private val cipher: KeystoreCipher,
) {
    private val key = stringPreferencesKey("refresh_token")

    suspend fun save(token: String) {
        val blob = cipher.encrypt(token.encodeToByteArray())
        val text = Base64.encodeToString(blob, Base64.NO_WRAP)
        dataStore.edit { it[key] = text }
    }

    suspend fun read(): String? =
        dataStore.data.first()[key]
            ?.let { Base64.decode(it, Base64.NO_WRAP) }
            ?.let { cipher.decrypt(it).decodeToString() }
}
```

INTERVIEW TRAP

"`EncryptedSharedPreferences` *is the recommended way."* — It was; last year's version still called it "modern and officially recommended". Google deprecated the whole library and points to platform APIs and the Keystore directly. Need more than one key or key rotation? Use Tink with a Keystore-wrapped keyset.

INTERVIEW TRAP

Keystore keys are **never backed up**. Restore the ciphertext on a new phone and decryption fails. Exclude it from backup (Q2) and treat `AEADBadTagException` as "clear and sign in again", not a crash.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Where secrets leak without any "hack"

Channel	Typical leak	Fix
Logcat	Tokens and personal data in `Log.d`	Strip logs in release; no-op logger
Crash & analytics SDKs	Emails, request bodies in breadcrumbs	Redact before sending; allow-list fields
HTTP logging	`Authorization` header in `BODY` logs	`Level.NONE` in release; `redactHeader()`
Backups	Refresh token restored elsewhere	`dataExtractionRules` (Q2)
Clipboard	Copied OTP shown in the clipboard preview	Mark it sensitive (API 33); clear it
Notifications	A lab result on the lock screen	`VISIBILITY_PRIVATE` + a public version
Screen & recents	OTP in screenshots	`FLAG_SECURE` (Day 20)

Last year's advice to zero out `CharArray` passwords still helps a little — but the runtime may already have copied them. The bigger wins are the channels above.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Logging safely

```
# proguard-rules.pro — drop verbose logs in release
-assumenosideeffects class android.util.Log {
    public static int v(...);
    public static int d(...);
}
```

```
val logging = HttpLoggingInterceptor().apply {
    level = if (BuildConfig.DEBUG) Level.BODY else Level.NONE
    redactHeader("Authorization")
    redactHeader("Cookie")
}
```

INTERVIEW TRAP

"Nobody sees `Log.d` in release." — Anyone with USB debugging runs `adb logcat`, and logs end up in bug reports people attach to tickets. If it's secret, don't log it — in any build type.

PLANT, DON'T SPRINKLE

Route all logging through one logger (e.g. Timber with a tree planted only in debug), so "no secrets in logs" is one decision, not a thousand call sites.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 19

Architect

Threat modeling · SDKs & the permission blast radius · Secure-by-default platform



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Threat modeling a feature in one hour



1. **Draw the data flow** and mark every trust boundary
2. **List the assets:** tokens, health data, money movements
3. **At each boundary crossing, walk STRIDE** (next slide)
4. **Decide:** mitigate, accept, or move the logic to the server — and track it as tickets

INTERVIEW TRAP

"The app enforces the limit, so the server can trust it." — Anything in the APK can be patched. Prices, limits, roles and feature entitlements are validated **on the server**; the client only mirrors them for UX.



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. STRIDE, in Android terms

Threat	Android example	Mitigation
Spoofing	A fake app calls your exported service	`signature` permission; check the caller
Tampering	Repackaged APK, hooked process	Server-side integrity checks (Day 20)
Repudiation	"I never sent that transfer"	Server audit log; signed requests (Day 20)
Information disclosure	Tokens in logs or backups	Q9 · Q10
Denial of service	Crash from malformed extras	Validate every intent input
Elevation of privilege	Intent redirection, mutable `PendingIntent`	Q5 · Q6

FOLLOW-UP

"Who owns the threat model?" — The feature team, with a security champion reviewing. It lives next to the design doc and is updated when the data flow changes — not written once for an audit.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. SDKs and the permission blast radius

Every SDK runs **in your process, with your UID and all your permissions**, and its manifest is **merged into yours**.

```
<manifest xmlns:tools="http://schemas.android.com/tools">
    <!-- An SDK asked for it; no feature of ours needs it -->
    <uses-permission
        android:name="android.permission.READ_PHONE_STATE"
        tools:node="remove" />
</manifest>
```

```
# CI: fail when the release APK's permission list changes
apkanalyzer manifest permissions app-release.apk > perms.txt
diff perms.baseline.txt perms.txt
```

- `QUERY_ALL_PACKAGES` is restricted by Play policy — declare narrow `<queries>` instead
- Google Play SDK Index flags SDK versions with known policy or security issues



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. What interviewers check

INTERVIEW TRAP

"What the analytics SDK collects is the vendor's problem." — Not on Google Play: your **Data safety** form must declare the data your SDKs collect. Their leak is your policy violation.

INTERVIEW TRAP

"We only added a library, not a permission." — The manifest merger added it for you. Review the **merged manifest** and the release APK, not just your own `AndroidManifest.xml`.

FOLLOW-UP

"How do you evaluate a new SDK?" — Permissions and components it adds; what it sends over the network (inspect the traffic); whether init can wait for user consent; maintenance and CVE history; its SDK Index status.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Secure by default: make the safe path the easy one

```
// build-logic convention plugin, applied to every app module
extensions.configure<ApplicationExtension> {
    lint {
        error += setOf(
            "SetJavaScriptEnabled",
            "AddJavascriptInterface",
            "ExportedService",
            "UnspecifiedImmutableFlag",
            "TrustAllX509TrustManager",
            "HardcodedDebugMode",
        )
        abortOnError = true
        checkReleaseBuilds = true
    }
}
```

Lint runs on every pull request, so an insecure default fails the build instead of waiting for a pentest.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. The paved road

Provide centrally	So teams don't hand-roll
`:core:security` — `KeystoreCipher`, `TokenStore`	Crypto with the wrong mode or a fixed IV
A `SecureWebView` factory with allow-listed origins	Per-screen WebView settings
Helpers that build explicit, immutable intents	Mutable implicit `PendingIntent`s
Lint rules in CI, StrictMode in debug	Enforcement by code review only
A threat-model template in the design doc	Security findings a week before release

MY TAKE

Security teams don't scale by reviewing every PR. They scale by shipping a paved road — and letting the build say "no" to the unpaved one.

INTERVIEW TRAP

"Security is a pentest before the big release." — Findings that late are expensive and get "accepted". Threat model at design time, lint at PR time, pentest to **verify**.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 19 · ARCHITECT

Day 19 — cheat sheet

- The sandbox isolates files, not entry points: exported components, intents, WebViews, logs and backups are all doors.
- Least privilege: request permissions in context; the Photo Picker needs **no** permission.
- After two denials (Android 11+) the permission dialog stops appearing — explain, then link to settings.
- Declare `exported` explicitly; filters aren't access control; never forward an embedded `Intent` blindly.
- `PendingIntent`: explicit + `FLAG_IMMUTABLE` by default; extras don't make tokens distinct.
- WebView: JavaScript only if needed, no file access, allow-listed hosts, `addWebMessageListener` over `addJavascriptInterface`.
- `security-crypto` is deprecated: Keystore key + AES-GCM + DataStore/files; exclude ciphertext from backup.
- StrongBox throws when missing; Keystore protects key bytes, not key use.
- Threat model per feature; audit SDK permissions; enforce defaults with Lint in CI.

Day 20 is next → Security, part 2: R8, integrity, biometrics & passkeys. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700